CS 182 - HW 3 - Jie Wen

1. Maximal Update Parameterization

(a) Expected Squared RMS Norm of Output Features

Consider a dense layer with input $x \in \mathbb{R}^{d_1}$ and weights $W \in \mathbb{R}^{d_2 \times d_1}$, both initialized with i.i.d. standard Gaussian entries. The output is $y = Wx$.

The squared RMS norm of y is defined as:

$$\text{RMS}(y)^2 = \frac{1}{d_2}\|y\|_2^2 = \frac{1}{d_2}\sum_{j=1}^{d_2} y_j^2.$$

Since W and x are independent with i.i.d. Gaussian entries (mean 0, variance 1), each output component $y_j = W_j x$ (where W_j is the j-th row of W) has mean 0 and variance:

$$\text{Var}(y_j) = \sum_{k=1}^{d_1} \text{Var}(W_{j,k})\text{Var}(x_k) = d_1 \cdot 1 \cdot 1 = d_1.$$

Thus, $\mathbb{E}[y_j^2] = d_1$.

The expected squared RMS norm is:

$$\mathbb{E}[\text{RMS}(y)^2] = \frac{1}{d_2}\sum_{j=1}^{d_2} \mathbb{E}[y_j^2] = \frac{1}{d_2} \cdot d_2 \cdot d_1 = d_1.$$

Thus, to ensure $\mathbb{E}[\text{RMS}(y)^2] = 1$, scale W by $\frac{1}{\sqrt{d_1}}$.

(b) Expected RMS Norm of Feature Change via SignGD

The SignGD update rule is:

$$W_{t+1} \leftarrow W_t + \eta \cdot \text{sign}(x_i g_i^T),$$

where $x_i \in \mathbb{R}^{d_1}$ is the input and $g_i \in \mathbb{R}^{d_2}$ is the backpropagated vector. The unscaled change in features is:

$$y' = \text{sign}(x_i g_i^T) x_i.$$

Note that:

$$\text{sign}(x_i g_i^T) = \text{sign}(g_i)\text{sign}(x_i)^T,$$

so:

$$y' = \text{sign}(g_i) \cdot (\text{sign}(x_i)^T x_i).$$

The scalar $\text{sign}(x_i)^T x_i = \sum_{k=1}^{d_1} \text{sign}((x_i)_k) \cdot (x_i)_k = \sum_{k=1}^{d_1} |(x_i)_k|$, since $\text{sign}(a) \cdot a = |a|$.

Thus:

$$y' = \left(\sum_{k=1}^{d_1} |(x_i)_k|\right) \cdot \text{sign}(g_i).$$

The squared RMS norm of y' is:

$$\text{RMS}(y')^2 = \frac{1}{d_2}\|y'\|_2^2 = \frac{1}{d_2}\left(\sum_{k=1}^{d_1}|(x_i)_k|\right)^2 \cdot \|\text{sign}(g_i)\|_2^2.$$

Since $\|\text{sign}(g_i)\|_2^2 = d_2$:

$$\text{RMS}(y')^2 = \left(\sum_{k=1}^{d_1}|(x_i)_k|\right)^2.$$

Each $|(x_i)_k|$ is a half-normal random variable with mean $\sqrt{2/\pi}$ and variance $1 - 2/\pi$. Thus:

$$\mathbb{E}\left[\sum_{k=1}^{d_1}|(x_i)_k|\right] = d_1\sqrt{\frac{2}{\pi}}, \quad \text{Var}\left(\sum_{k=1}^{d_1}|(x_i)_k|\right) = d_1\left(1 - \frac{2}{\pi}\right).$$

So:

$$\mathbb{E}[\text{RMS}(y')^2] = \mathbb{E}\left[\left(\sum_{k=1}^{d_1}|(x_i)_k|\right)^2\right] = \text{Var} + (\mathbb{E})^2 = d_1\left(1 - \frac{2}{\pi}\right) + d_1^2 \cdot \frac{2}{\pi}.$$

Thus, to ensure $\mathbb{E}[\text{RMS}(y')^2] = 1$, scale the update by:

$$c = \frac{1}{\sqrt{d_1\left(1 - \frac{2}{\pi}\right) + d_1^2 \cdot \frac{2}{\pi}}}.$$

For large d_1, this simplifies to $c \approx \frac{\sqrt{\pi/2}}{d_1}$.

(c) Why the Update Depends Only on d_1

The RMS norm of the feature change y' reduces to:

$$\text{RMS}(y') = \left|\sum_{k=1}^{d_1}|(x_i)_k|\right|,$$

which is independent of d_2. This occurs because:

- The update matrix $\text{sign}(x_i g_i^T)$ is rank-1.
- The resulting feature change y' is proportional to $\text{sign}(g_i)$, whose norm scales with d_2, but this scaling is canceled by the RMS normalization (division by $\sqrt{d_2}$).

Thus, the RMS norm of y' depends only on the input dimension d_1.

---

2. Visualizing Maximal Update Parameterization (Coding Question)

Maximal Update Parameterization

In this problem, we will examine the training of a simple MLP with hidden layers of varying widths. We will then investigate the maximal update parameterization (muP) which will allow us to use a single global learning rate to jointly train layers of any width.

Note: This homework question is new this year and it is messier than usual. We felt it was worth it to get it out so you can play with these new techniques. If you're feeling stuck, don't hesistate to ask questions on Ed.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from keras.datasets import mnist
import matplotlib.pyplot as plt

%matplotlib inline

# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class MLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes = [8, 16, 32, 64,
128], num_classes=10):
        super(MLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        for i in range(len(all_hidden_sizes)-1):
            self.layers.append(nn.Linear(all_hidden_sizes[i],
all_hidden_sizes[i+1]))
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        activations = []
        x = x.view(x.size(0), -1)  # Flatten: (batch_size, 28*28)
        for layer in self.layers[:-1]:
            x = self.sigmoid(layer(x))
            activations.append(x)
        x = self.layers[-1](x)
        activations = activations[1:]
        return x, [a.detach() for a in activations]

# Load MNIST data
(train_images, train_labels), (valid_images, valid_labels) =
mnist.load_data()

# Normalize pixel values to [0, 1]
train_images = train_images.astype(np.float32) / 255.0
```

```
valid_images = valid_images.astype(np.float32) / 255.0

# Convert to PyTorch tensors
train_images = torch.from_numpy(train_images)
train_labels = torch.from_numpy(train_labels).long()
valid_images = torch.from_numpy(valid_images)
valid_labels = torch.from_numpy(valid_labels).long()

def rms(x, dim):
    return torch.sqrt(torch.mean(x**2, dim=dim))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleAdam(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdam, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group['params']:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
                    state['momentum'] = torch.zeros_like(p)
                    state['variance'] = torch.zeros_like(p)

                state['step'] += 1
                m = state['momentum']
                m.lerp_(grad, 1-group["b1"])
                v = state['variance']
                v.lerp_(grad**2, 1-group["b2"])

                m_hat = m / (1 - group["b1"]**state['step'])
                v_hat = v / (1 - group["b2"]**state['step'])
                u = m_hat / (torch.sqrt(v_hat) + 1e-16)
```

```
                p.add_(u, alpha=-group['lr'])
        return None

batch_idx = np.random.randint(0, len(train_images), size=64)
def train_one_step(mlp=MLP, hiddens=[8, 16, 64, 64, 64, 256, 256,
1024], optimizer=SimpleAdam, label="Adam", lr=0.01):
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

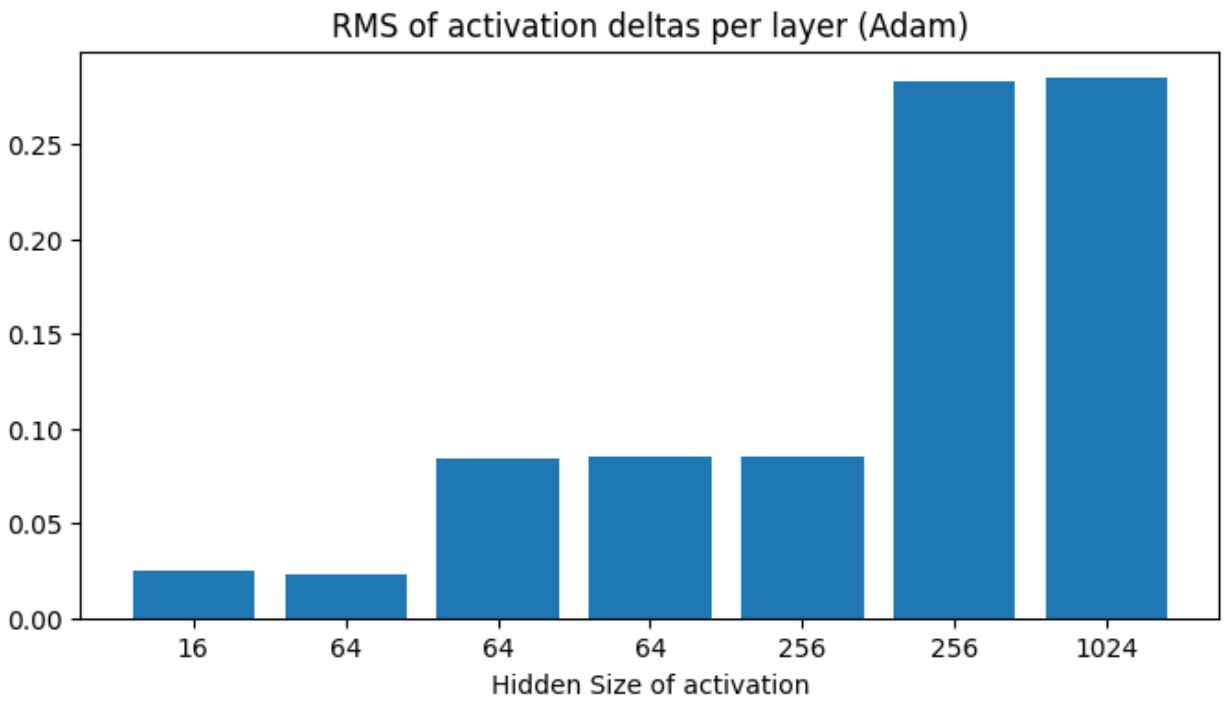
    prev_activations = None
    for i in range(2):
        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device),
labels_batch.to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

        if i > 0:
            print([a.shape for a in activations])
            activation_deltas = [a - pa for a, pa in zip(activations,
prev_activations)]
            activation_deltas_rms = [torch.mean(rms(a, dim=-1)) for a
in activation_deltas]
        prev_activations = activations

    # plot deltas
    deltas = np.array(activation_deltas_rms)
    fig, axs = plt.subplots(1, figsize=(8, 4))
    axs.set_title(f'RMS of activation deltas per layer ({label})')
    axs.set_xlabel('Hidden Size of activation')
    axs.bar(np.arange(deltas.shape[0]), deltas)
    axs.set_xticks(np.arange(deltas.shape[0]))
    axs.set_xticklabels(hiddens[1:])
    plt.show()
train_one_step(optimizer=SimpleAdam)
```

```
[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),
torch.Size([64, 1024])]
```



a. Examining the norms of a heterogenous MLP.

Run the above cell, which trains a neural network for a single gradient step, then examines the effect of that step on the resulting activations. What are the dimensions of each layer in the neural network?

Answer: Input: 784, Hidden: [8, 16, 64, 64, 64, 256, 256, 1024], Output: 10

How does the dimensionality of the layer affect the RMS norm of the activation deltas?

Answer: Wider layers have larger RMS activation deltas because they accumulate updates from more parameters

---

Change the widths of some of your neural network layers, and recreate the plot -- did the RMS values change as expected?

Answer: Yes, increasing layer width increases RMS values as expected

```
# TODO: Call some plotting code here.
```

b. Examining the norms of the updates to the weights.

In the provided code above, we plotted the change in norms of the *activation vectors*. Now, you will examine the change in the weights themselves. Create a version of the above function that runs a single gradient step, then for each dense layer plot:

- The *Frobenius* norm of the update.
- The *spectral* norm of the update.
- The *RMS-RMS induced norm* of the update.

Which one of these norms correlates the most with the RMS norms of the activations?

Answer: The RMS-RMS induced norm correlates most with the RMS norms of the activations because it accounts for both the input and output dimensions of each layer, similar to how activations flow through the network.

You should calculate your updates as `new_dense_parameter - old_dense_parameter`.

```
### Solution
batch_idx = np.random.randint(0, len(train_images), size=64)
def train_one_step_matrices(mlp=MLP, hiddens=[8, 16, 64, 64, 64, 256,
256, 1024], optimizer=SimpleAdam, label="Adam", lr=0.01):
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

    old_params = [p.detach().clone() for p in model.parameters()]

    for i in range(1):
        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device),
labels_batch.to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

    new_params = [p.detach().clone() for p in model.parameters()]
    delta_params = [new_p - old_p for new_p, old_p in zip(new_params,
old_params)]

    frob_norms = []
    spectral_norms = []
    induced_norms = []
    p_shapes = []

    for p in delta_params:
```

```
        if len(p.shape) == 2:
            ### TODO: Log the respective norms.
            # Frobenius norm
            frob_norm = torch.norm(p, p='fro').item()
            frob_norms.append(frob_norm)

            # Spectral norm (largest singular value)
            u, s, vh = torch.linalg.svd(p, full_matrices=False)
            spectral_norm = s[0].item()
            spectral_norms.append(spectral_norm)

            # RMS-RMS induced norm: spectral_norm *
sqrt(input_dim/output_dim)
            input_dim = p.shape[1]
            output_dim = p.shape[0]
            induced_norm = spectral_norm * np.sqrt(input_dim /
output_dim)
            induced_norms.append(induced_norm)

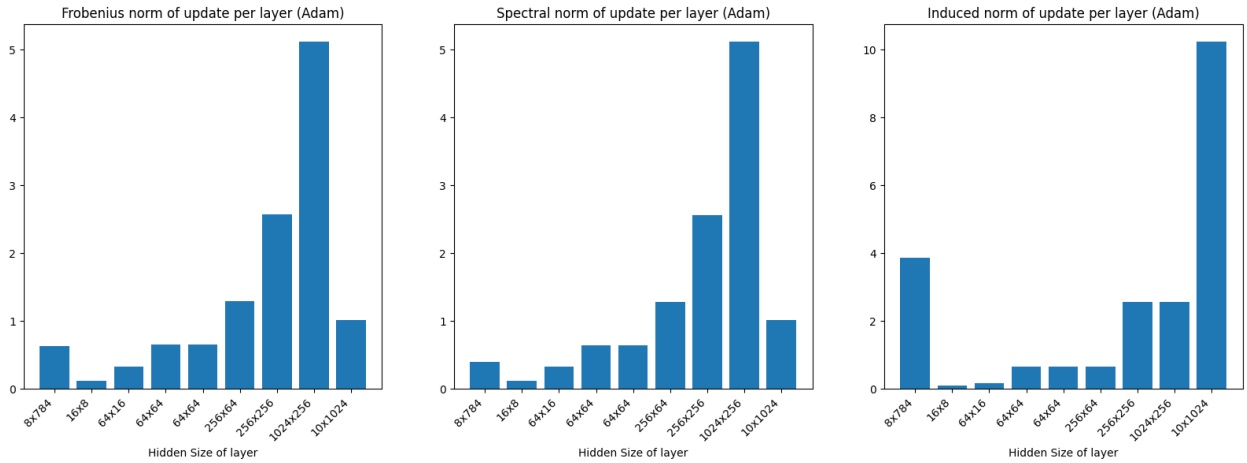
            p_shapes.append(f"{p.shape[0]}x{p.shape[1]}")
            ###


    fig, axs = plt.subplots(1, 3, figsize=(20, 6))

    axs[0].set_title(f'Frobenius norm of update per layer ({label})')
    axs[0].set_xlabel('Hidden Size of layer')
    axs[0].bar(np.arange(len(frob_norms)), frob_norms)
    axs[0].set_xticks(np.arange(len(frob_norms)), p_shapes,
rotation=45, ha='right')

    axs[1].set_title(f'Spectral norm of update per layer ({label})')
    axs[1].set_xlabel('Hidden Size of layer')
    axs[1].bar(np.arange(len(spectral_norms)), spectral_norms)
    axs[1].set_xticks(np.arange(len(spectral_norms)), p_shapes,
rotation=45, ha='right')

    axs[2].set_title(f'Induced norm of update per layer ({label})')
    axs[2].set_xlabel('Hidden Size of layer')
    axs[2].bar(np.arange(len(induced_norms)), induced_norms)
    axs[2].set_xticks(np.arange(len(induced_norms)), p_shapes,
rotation=45, ha='right')
    plt.show()
train_one_step_matrices(optimizer=SimpleAdam)
```



c. Implementing muP

We will now implement muP scaling. Modify the starter code below to set a per-layer learning rate such that the resulting RMS activation-deltas are uniform scale, regardless of the layer widths. Plot the resulting activation-deltas on at least two sets of widths.

Note: Even with the correct scaling, the first 2-3 activation-deltas may have a lower norm than the rest. Can you think of a reason why this might be the case?

```
from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleAdamMuP(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdamMuP, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        layer_idx = 0
        total_layers = sum(1 for group in self.param_groups for p in
group['params'] if len(p.shape) == 2)

        for group in self.param_groups:
            for p in group['params']:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
```

```
                    state['momentum'] = torch.zeros_like(p)
                    state['variance'] = torch.zeros_like(p)

                state['step'] += 1
                m = state['momentum']
                m.lerp_(grad, 1-group["b1"])
                v = state['variance']
                v.lerp_(grad**2, 1-group["b2"])

                m_hat = m / (1 - group["b1"]**state['step'])
                v_hat = v / (1 - group["b2"]**state['step'])
                u = m_hat / (torch.sqrt(v_hat) + 1e-16)

                ############################
                ### Todo: Adjust the per-layer learning rate scaling
factor so per-layer RMS activation deltas are constant.
                ### Hint for part e: The following tricks will help
you retain performance when using muP scaling.
                ###  - Treat biases as a hidden layer with size
(d_out, 1). You will need to use a fudge-factor of around 0.01 -- we
want to keep the change in bias terms low.
                ###  - For the input layer, a fudge factor of 10
appears to help.
                ###  - For the output layer, we find it is best to
ignore the muP scaling, and instead use a fixed learning rate (e.g.
0.003).
                ############################
                lr = group['lr']
                if len(p.shape) == 2:  # Weight matrix
                    input_dim = p.shape[1]
                    output_dim = p.shape[0]

                    if layer_idx == 0:  # First layer
                        lr_scale = 10.0 / np.sqrt(input_dim)
                    elif layer_idx == total_layers - 1:  # Last layer
                        lr_scale = 0.003 / lr  # Fixed learning rate
for output layer
                    else:  # Hidden layers
                        lr_scale = 1.0 / np.sqrt(input_dim)

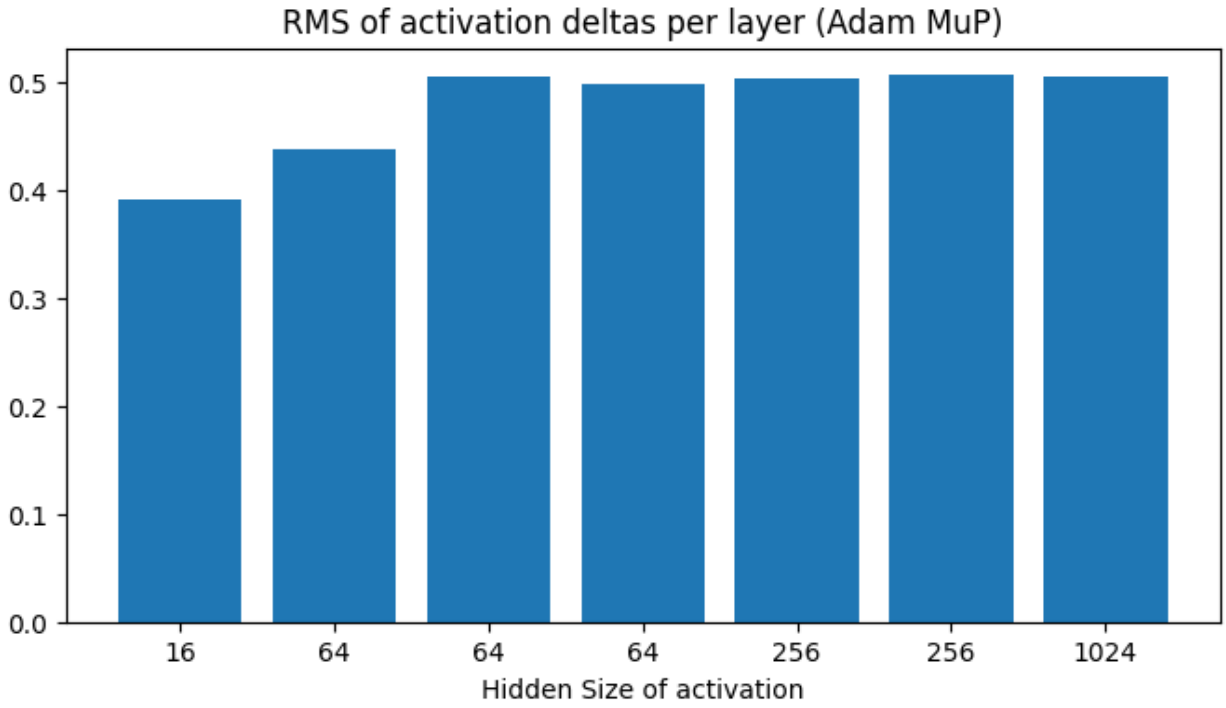
                    layer_idx += 1
                else:  # Bias term
                    lr_scale = 0.01  # Fudge factor for biases

                lr *= lr_scale
                ############################
                ############################

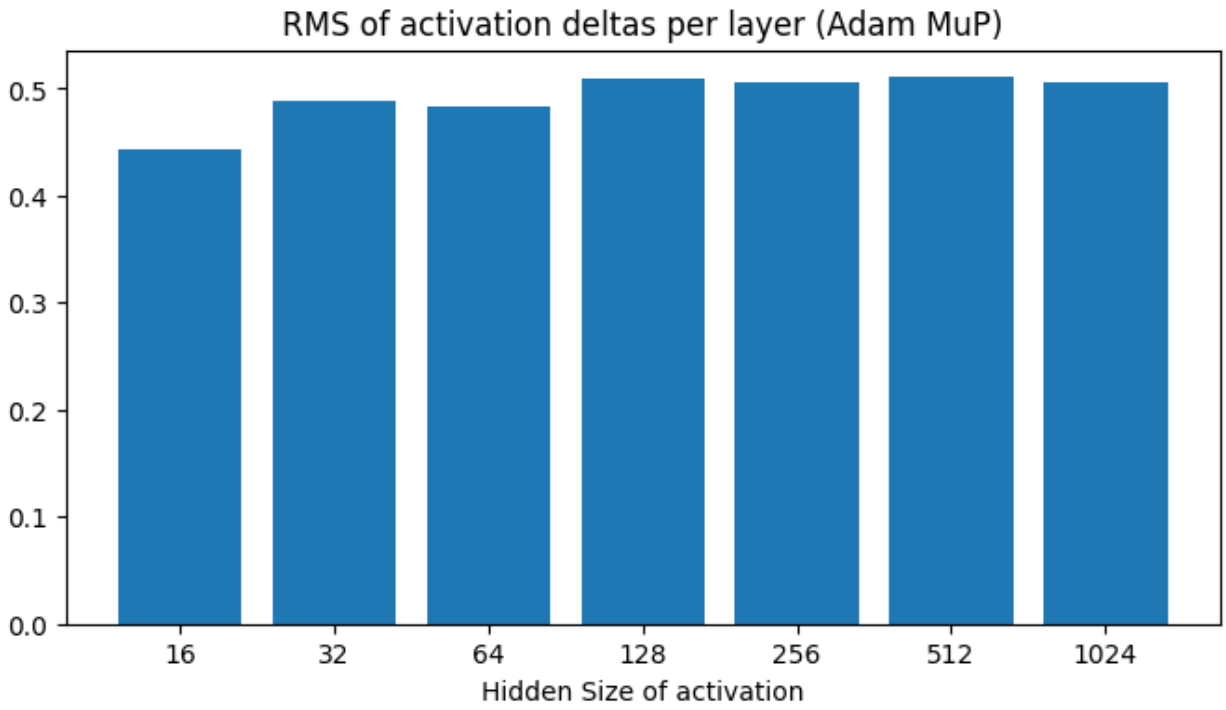
                p.add_(u, alpha=-lr)
        return None
```

```
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP",
hiddens=[8, 16, 64, 64, 64, 256, 256, 1024])
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP",
hiddens=[8, 16, 32, 64, 128, 256, 512, 1024])
```

```
[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),
torch.Size([64, 1024])]
```




```
[torch.Size([64, 16]), torch.Size([64, 32]), torch.Size([64, 64]),
torch.Size([64, 128]), torch.Size([64, 256]), torch.Size([64, 512]),
torch.Size([64, 1024])]
```



d. Per-Weight Multipliers

An alternative way to implement muP is to adjust the *network graph* itself, rather than the optimizer. Implement this below, and recreate the above uniformly-scaled graph when using the *Adam* (not muP) optimizer. We have disabled biases to simplify the problem.

Why is multiplying the output of a layer by a constant the same as adjusting the learning-rate of that layer (when using Adam or SignGD)?

Answer: Multiplying the output of a layer by a constant is equivalent to adjusting the learning rate because both operations scale the gradient flow through that layer. In Adam/SignGD, the update direction is normalized, so scaling the forward pass has a similar effect to scaling the learning rate.

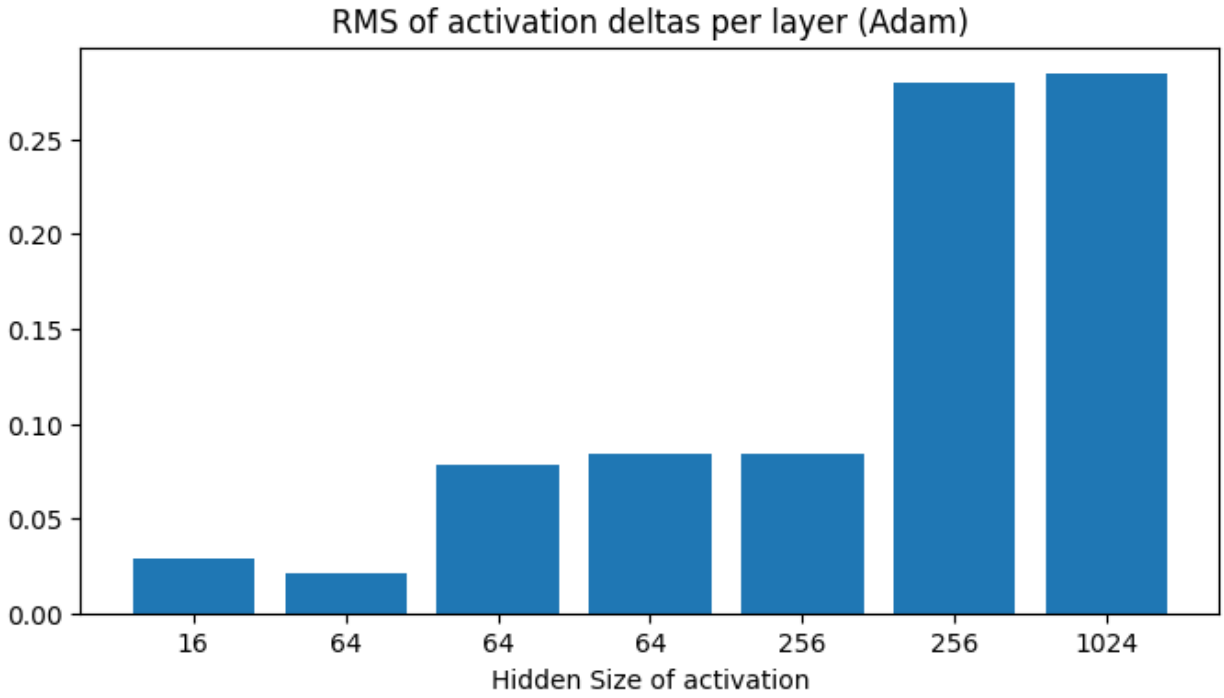
```
class ScaledMLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes = [8, 16, 32, 64,
128], num_classes=10):
        super(ScaledMLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        for i in range(len(all_hidden_sizes)-1):
            self.layers.append(nn.Linear(all_hidden_sizes[i],
all_hidden_sizes[i+1], bias=False))
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        activations = []
```

```
        x = x.view(x.size(0), -1)  # Flatten: (batch_size, 28*28)
        for layer in self.layers[:-1]:
            x = layer(x)
            ## TODO
            pass
            ##
            x = self.sigmoid(x)
            activations.append(x)
        x = self.layers[-1](x)
        activations = activations[1:]
        return x, [a.detach() for a in activations]

train_one_step(mlp=ScaledMLP, optimizer=SimpleAdam)
```

```
[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),
torch.Size([64, 1024])]
```



e. Hyperparameter Transfer

Run the following code, which will perform a sweep over learning rates for 3-layer MLPs of increasing width using Adam. How does the optimal learning rate change as the network increases in size?

Answer: As the network increases in size, the optimal learning rate for Adam decreases. This is because the magnitude of the gradients tends to increase with the width of the network, and a smaller learning rate is needed to prevent the training from becoming unstable.

In the second cell, we will instead use the muP optimizer you implemented. How does the optimal learning rate work now? You should aim to show that there is a single global learning rate that works on a majority of widths. The 256-width network should achieve a loss of 0.5, comparable to Adam.

Answer: With the muP optimizer, the optimal learning rate is much more stable across different network widths. A single global learning rate can be found that works well for a majority of the widths. This is because muP is designed to make the training dynamics independent of the model size, which allows for the transfer of hyperparameters like the learning rate from smaller to larger models.

```
valid_idx = np.random.randint(0, len(train_images), size=64)
valid_images = train_images[valid_idx]
valid_labels = train_labels[valid_idx]
valid_images, valid_labels = valid_images.to(device),
valid_labels.to(device)


def train_with_lr(hiddens=[64, 64, 64], optimizer=SimpleAdam,
lr=0.01):
    torch.manual_seed(4)
    np.random.seed(4)
    model = MLP(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)
    losses = []

    for i in range(100):
        batch_idx = np.random.randint(0, len(train_images), size=64)

        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device),
labels_batch.to(device)

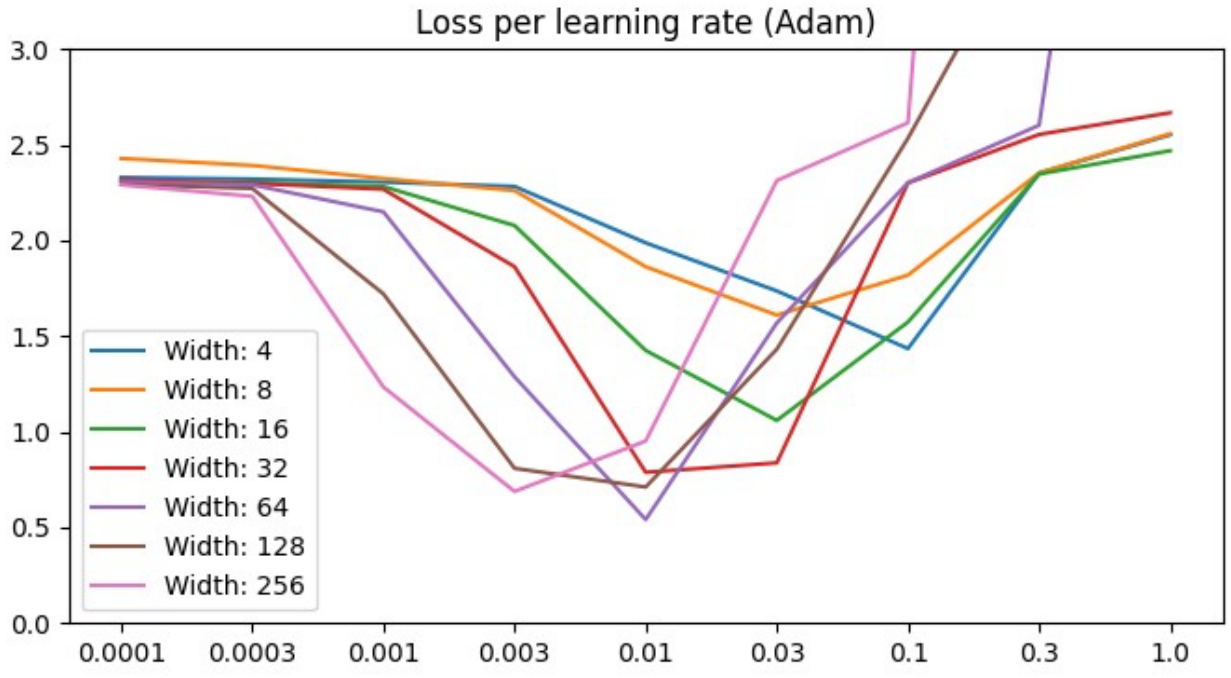
        optimizer.zero_grad()
        outputs, _ = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

        with torch.no_grad():
            outputs_valid, _ = model(valid_images)
            valid_losses = criterion(outputs_valid, valid_labels)
            losses.append(valid_losses.item())

    return np.mean(np.array(losses)[-5:])
```

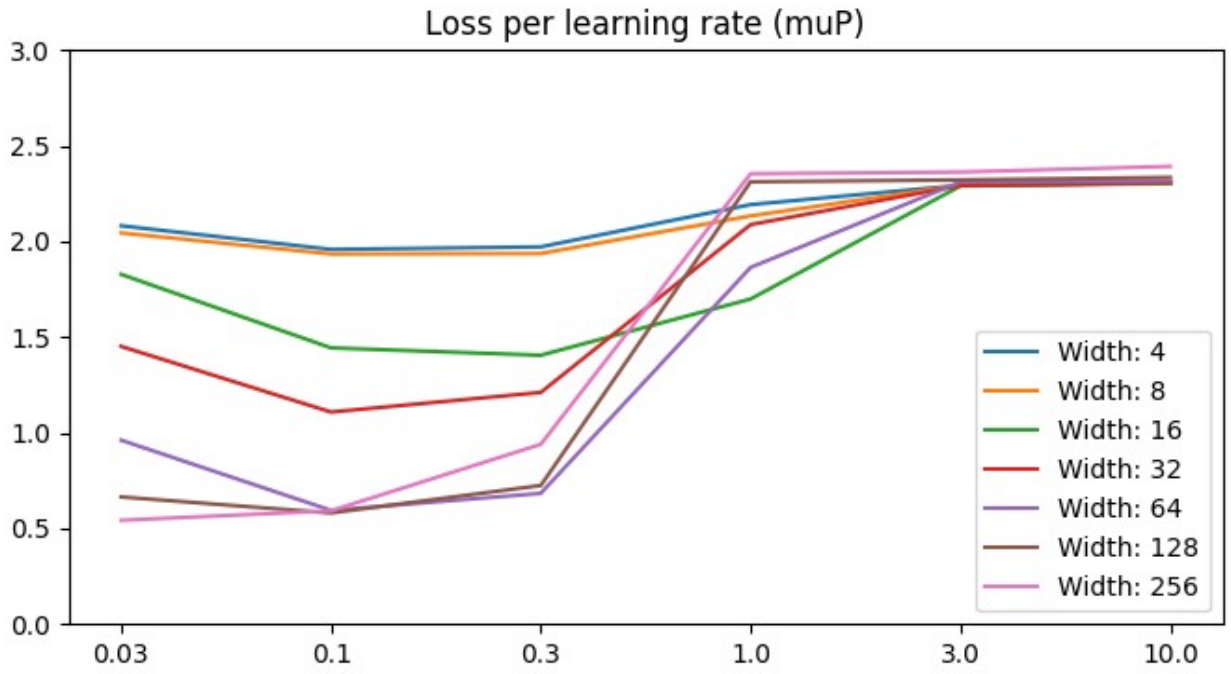
```
all_widths = [4, 8, 16, 32, 64, 128, 256]
all_lrs = [0.0001, 0.0003, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0]
adam_results = np.zeros((len(all_widths), len(all_lrs)))
for wi, width in enumerate(all_widths):
    for lri, lr in enumerate(all_lrs):
        loss = train_with_lr(hiddens=[width, width, width], lr=lr)
        adam_results[wi, lri] = loss

fig, axs = plt.subplots(1, figsize=(8, 4))
axs.set_title(f'Loss per learning rate (Adam)')
for wi, width in enumerate(all_widths):
    axs.plot(np.arange(len(all_lrs)), adam_results[wi], label=f'Width:
{width}')
axs.set_xticks(np.arange(len(all_lrs)))
axs.set_xticklabels(all_lrs)
axs.set_ylim(bottom=0, top=3)
axs.legend()
fig.show()
```




```
all_widths = [4, 8, 16, 32, 64, 128, 256]
all_lrs = [0.03, 0.1, 0.3, 1.0, 3.0, 10.0]
mup_results = np.zeros((len(all_widths), len(all_lrs)))
for wi, width in enumerate(all_widths):
    for lri, lr in enumerate(all_lrs):
        loss = train_with_lr(hiddens=[width, width, width], lr=lr,
optimizer=SimpleAdamMuP)
        mup_results[wi, lri] = loss
```

```
fig, axs = plt.subplots(1, figsize=(8, 4))
axs.set_title(f'Loss per learning rate (muP)')
for wi, width in enumerate(all_widths):
    axs.plot(np.arange(len(all_lrs)), mup_results[wi], label=f'Width:
{width}')
axs.set_xticks(np.arange(len(all_lrs)))
axs.set_xticklabels(all_lrs)
axs.set_ylim(bottom=0, top=3)
axs.legend()
fig.show()
```



f. Shampoo and Orthogonalization

In lecture, we discussed a simplified version of the Shampoo update, which can be viewed as *orthogonalizing* the update to a dense layer. In the following code block, implement this simplified Shampoo update:

$$momentum \to U \Sigma V^T . update = U V^T .$$

Feel free to use linear algebra functions such as `torch.linalg.svd`.

```
from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleShampoo(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
```

```
        b1: float = 0.9,
    ):
        defaults = dict(lr=lr, b1=b1)
        super(SimpleShampoo, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group['params']:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
                    state['momentum'] = torch.zeros_like(p)

                state['step'] += 1
                m = state['momentum']
                m.lerp_(grad, 1-group["b1"])

                ############ TODO
                if len(m.shape) == 1:
                    u = m # Ignore biases for this question, it's not
important.
                else:
                    # SVD decomposition
                    U, S, Vh = torch.linalg.svd(m,
full_matrices=False)
                    # Orthogonal update: U * V^T
                    u = U @ Vh

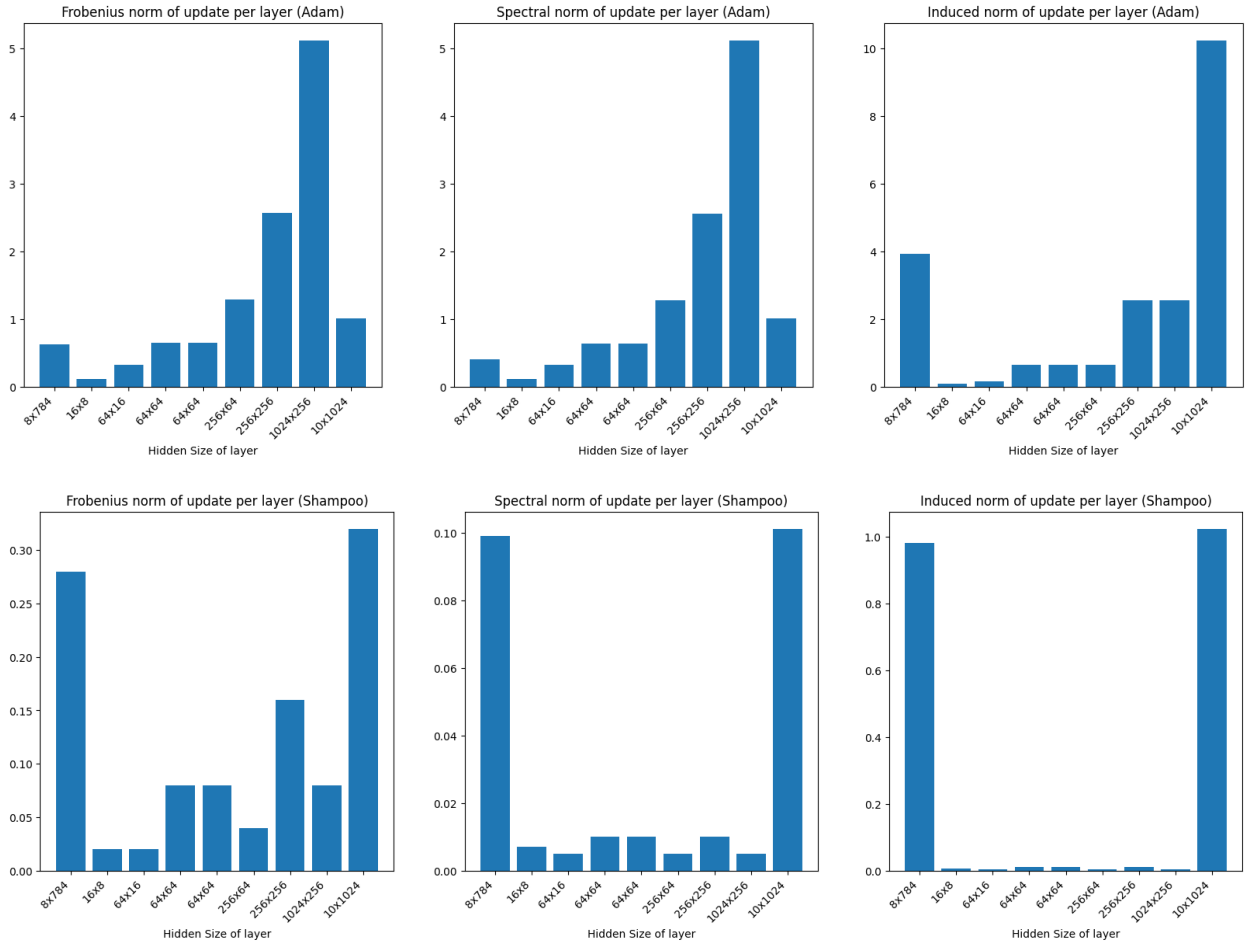
                    # Bonus: Scale to match RMS-RMS induced norm
                    input_dim = p.shape[1]
                    output_dim = p.shape[0]
                    scale_factor = np.sqrt(input_dim / output_dim)
                    u = u * scale_factor
                #############
                p.add_(u, alpha=-group['lr'])
        return None
```

Now, we will examine the relationship between the Frobenius norm and the Spectral norm for Adam vs. Shampoo. Plot these norms using your code from part c. What relationship do you see? Can you come up for a reason why this makes sense?

Answer: Shampoo produces updates with more uniform spectral norms across layers because it orthogonalizes the update direction. The Frobenius norm still varies with layer dimensions, but the spectral norm (which determines the maximum scaling effect) becomes more consistent. This makes sense because Shampoo explicitly constructs an orthogonal update that preserves relative magnitudes across different directions.

Bonus: How should we scale the Shampoo update so the *induced RMS-RMS norm* is equal? Implement this change.

```
train_one_step_matrices(optimizer=SimpleAdam)
train_one_step_matrices(optimizer=SimpleShampoo, label="Shampoo")
```



3. Maximal Update Parameterization Research

Here are the answers to the homework problems, based on the provided research papers.

(a) Figure 1 of Paper #1

The two side-by-side figures in Figure 1 of "Tensor Programs V" are both plotting the **training loss versus the base-2 logarithm of the learning rate** for Transformer models of varying widths (from 128 to 8192). The plot on the left shows the results using the **Standard Parameterization (SP)**, while the plot on the right shows the results using the **Maximal Update Parameterization (μ P)**, referred to as "Our Work".

The key observation to make about the width of the network is how it affects the optimal learning rate.

- In the **Standard Parameterization (left)**, the optimal learning rate is unstable and **shifts to lower values as the network width increases**. Furthermore, wider networks do not consistently perform better; for a fixed learning rate, performance can degrade with width, and even with optimal tuning, the widest network (8192) is outperformed by its μ P counterpart.
- In the **Maximal Update Parameterization (right)**, the optimal learning rate is **stable across all tested widths**. The plot shows that a single learning rate is nearly optimal for all model sizes. Additionally, in μ P, wider models consistently achieve better (lower) training loss, demonstrating a "wider is better" principle.

(b) Table 3 of Paper #1

In Table 3, for hidden layers trained with Adam, the Maximal Update Parameterization (μ P) recommends a learning rate scaling of $1/\text{fan}_{\text{in}}$. The table indicates that the corresponding **Standard Parameterization (SP)**, shown in gray text, uses a scaling of **1**. This means that in standard practice, the learning rate for hidden layers is not scaled with the layer's input dimension (`fan_in`).

(c) Desideratum 1 and Condition 1 of Paper #2

- **Why Desideratum 1 is desired for feature learning:**
  Desideratum 1, which states that $\|h_l\|_2 = \Theta(\sqrt{n_l})$ and $\|\Delta h_l\|_2 = \Theta(\sqrt{n_l})$, is desired because it ensures that the features and their updates during training are stable and non-trivial as network width (n_l) increases. The paper explains this with two points:
  1. **Stable Activations:** The condition $\|h_l\|_2 = \Theta(\sqrt{n_l})$ means that the "typical element size" of the feature vector is $\Theta(1)$. This is crucial because activation functions are designed to operate on inputs that are of order one to produce meaningful, non-saturating outputs.
  2. **Nontrivial Updates:** The condition $\|\Delta h_l\|_2 = \Theta(\sqrt{n_l})$ ensures that the features change by a meaningful amount at each step. If the updates were larger, they would explode at large widths, leading to unstable training. If they were smaller, they would vanish, causing the features to stop changing and thus halting the learning process.
- **Rewriting Desideratum 1 and Condition 1:**
  The Root Mean Square (RMS) norm of a vector $v \in \mathbb{R}^n$ is defined as $\text{RMS}(v) = \|v\|_2/\sqrt{n}$.
  - **Desideratum 1** can be rewritten in terms of the RMS norm as:
    - $\text{RMS}(h_l) = \frac{\|h_l\|_2}{\sqrt{n_l}} = \frac{\Theta(\sqrt{n_l})}{\sqrt{n_l}} = \Theta(1)$
    - $\text{RMS}(\Delta h_l) = \frac{\|\Delta h_l\|_2}{\sqrt{n_l}} = \frac{\Theta(\sqrt{n_l})}{\sqrt{n_l}} = \Theta(1)$
      This shows that the desideratum is equivalent to requiring the features and their updates to have an average element size of order one.
  - **Condition 1** applies to matrices, for which an RMS norm is not standard. However, it can be interpreted as the RMS of the matrix entries, which is related to the Frobenius norm ($\|A\|_F$) by $\text{RMS}(A) = \|A\|_F/\sqrt{n_l n_{l-1}}$. Using the conversions in Table 1 of the paper, Condition 1 is equivalent to the following scalings for the RMS entry size:
    - $\text{RMS}(W_l) = \Theta\left(\sqrt{\frac{\min(n_l, n_{l-1})}{n_l n_{l-1}^2}}\right)$
    - $\text{RMS}(\Delta W_l) = \Theta\left(\frac{1}{n_{l-1}}\right)$
- **How Condition 1 implies upper bounds:**
  Condition 1 implies the upper bounds on feature and update norms through the submultiplicativity property of the spectral norm ($\|Av\|_2 \le \|A\|_* \|v\|_2$). The argument proceeds recursively:
  1. **For $\|h_l(x)\|_2$:** Assuming $\|h_{l-1}(x)\|_2 = \Theta(\sqrt{n_{l-1}})$ holds for the previous layer, we have:
     $\|h_l(x)\|_2 = \|W_l h_{l-1}(x)\|_2 \le \|W_l\|_* \|h_{l-1}(x)\|_2$.
     Substituting from Condition 1 ($\|W_l\|_* = \Theta(\sqrt{n_l/n_{l-1}})$) gives:
     $\|h_l(x)\|_2 \le \Theta(\sqrt{n_l/n_{l-1}}) \cdot \Theta(\sqrt{n_{l-1}}) = \Theta(\sqrt{n_l})$.
  2. **For $\|\Delta h_l(x)\|_2$:** The change in features is given by $\Delta h_l(x) = \Delta W_l h_{l-1}(x) + W_l \Delta h_{l-1}(x) + \Delta W_l \Delta h_{l-1}(x)$. Using the triangle inequality:
     $\|\Delta h_l(x)\|_2 \le \|\Delta W_l\|_* \|h_{l-1}(x)\|_2 + \|W_l\|_* \|\Delta h_{l-1}(x)\|_2 + \|\Delta W_l\|_* \|\Delta h_{l-1}(x)\|_2$.
     Substituting from Condition 1 and the recursive assumption gives:
     $\|\Delta h_l(x)\|_2 \le \Theta(\sqrt{n_l/n_{l-1}})\Theta(\sqrt{n_{l-1}}) + \Theta(\sqrt{n_l/n_{l-1}})\Theta(\sqrt{n_{l-1}}) + \cdots = \Theta(\sqrt{n_l})$.

- **Key assumption for lower bounds:**
  The key assumption that allows us to also get the lower bounds (and thus turn the $O(\cdot)$ bounds into $\Theta(\cdot)$ bounds) is the **alignment between input vectors and the top singular subspaces of the weight matrices and their updates**. The paper argues that during training, the input vectors $h_{l-1}(x)$ are not orthogonal to the largest singular vectors of W_l and ΔW_l. Specifically, for gradient updates (with batch size 1), the update matrix ΔW_l is rank-one and is perfectly aligned with its corresponding input vector $h_{l-1}(x)$. This alignment ensures that the submultiplicative inequality $\|Av\|_2 \leq \|A\|_* \|v\|_2$ becomes a tight scaling relationship, $\|Av\|_2 = \Theta(\|A\|_* \|v\|_2)$, which provides the necessary lower bound.

---

4. Policy Gradient and the Reparameterization Gradient Estimator

(a) Gradient with Respect to μ

$\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x}\sim p_\mu}\left[f(\mathbf{x}) \cdot \frac{\mathbf{x}-\mu}{\sigma^2}\right]$.
To arrive at this, start with $\mathcal{F}(\mu) = \int p_\mu(\mathbf{x}) f(\mathbf{x})\, d\mathbf{x}$. Differentiating with respect to μ (assuming interchangeability of the gradient and integral) gives $\nabla_\mu \mathcal{F}(\mu) = \int (\nabla_\mu p_\mu(\mathbf{x})) f(\mathbf{x})\, d\mathbf{x}$. For the Gaussian, $\nabla_\mu \log p_\mu(\mathbf{x}) = \frac{\mathbf{x}-\mu}{\sigma^2}$, so $\nabla_\mu p_\mu(\mathbf{x}) = p_\mu(\mathbf{x}) \cdot \frac{\mathbf{x}-\mu}{\sigma^2}$. Substituting yields the expectation form.

(b) Reparameterization Using a Standard Normal

$\mathcal{F}(\mu) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[f(\mu+\sigma\mathbf{z})\right]$.
To arrive at this, use the transformation $\mathbf{x} = \mu + \sigma\mathbf{z}$ where $\mathbf{z} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$, which ensures $\mathbf{x} \sim p_\mu(\mathbf{x})$. The expectation over $\mathbf{x}$ then becomes the expectation over $\mathbf{z}$.

(c) Gradient via Reparameterization

Yes: $\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x}\sim p_{\mu=0}}\left[\nabla_{\mathbf{y}} f(\mathbf{y})\big|_{\mathbf{y}=\mu+\mathbf{x}}\right]$.
To arrive at this, start from the reparameterized form in (b): $\mathcal{F}(\mu) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[f(\mu+\sigma\mathbf{z})\right]$. Differentiating with respect to μ (moving the gradient inside the expectation, as the distribution over $\mathbf{z}$ is independent of μ) gives $\nabla_\mu \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\nabla_\mu f(\mu+\sigma\mathbf{z})\right]$. By the chain rule, $\nabla_\mu f(\mu+\sigma\mathbf{z}) = \nabla_{\mathbf{y}} f(\mathbf{y})\big|_{\mathbf{y}=\mu+\sigma\mathbf{z}} \cdot \nabla_\mu(\mu+\sigma\mathbf{z}) = \nabla_{\mathbf{y}} f(\mathbf{y})\big|_{\mathbf{y}=\mu+\sigma\mathbf{z}}$, since $\nabla_\mu(\mu+\sigma\mathbf{z}) = \mathbf{I}$. Noting that $\mathbf{x} = \sigma\mathbf{z} \sim p_{\mu=0}(\mathbf{x}) = \mathcal{N}(\mathbf{0},\sigma^2\mathbf{I})$ and substituting $\mu+\sigma\mathbf{z} = \mu + \mathbf{x}$ yields the form.

(d) General Case — Score Function Estimator

Yes: $\nabla_\theta \mathcal{F}(\theta) = \mathbb{E}_{\mathbf{x}\sim p_\theta}\left[f(\mathbf{x}) \cdot \nabla_\theta \log p_\theta(\mathbf{x})\right]$.
To arrive at this, start with $\mathcal{F}(\theta) = \int p_\theta(\mathbf{x}) f(\mathbf{x})\, d\mathbf{x}$. Differentiating with respect to θ gives $\nabla_\theta \mathcal{F}(\theta) = \int (\nabla_\theta p_\theta(\mathbf{x})) f(\mathbf{x})\, d\mathbf{x}$. Since $\nabla_\theta p_\theta(\mathbf{x}) = p_\theta(\mathbf{x}) \cdot \nabla_\theta \log p_\theta(\mathbf{x})$, substituting yields the expectation form. This generalizes (a), where $\nabla_\mu \log p_\mu(\mathbf{x}) = \frac{\mathbf{x}-\mu}{\sigma^2}$.

(e) General Case — Reparameterization Trick

Start with $\mathcal{F}(\theta) = \mathbb{E}_{\mathbf{x}\sim p_\theta}\left[f(\mathbf{x})\right]$. By the reparameterization assumption, $\mathbf{x} = g(\mathbf{z},\theta)$ where $\mathbf{z} \sim p(\mathbf{z})$ (independent of θ), so $\mathcal{F}(\theta) = \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\left[f(g(\mathbf{z},\theta))\right]$. Differentiating with respect to θ (moving the gradient inside the expectation, as $p(\mathbf{z})$ is independent of θ) gives $\nabla_\theta \mathcal{F}(\theta) = \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\left[\nabla_\theta f(g(\mathbf{z},\theta))\right]$. By the chain rule, $\nabla_\theta f(g(\mathbf{z},\theta)) = \nabla_{\mathbf{x}} f(\mathbf{x})\big|_{\mathbf{x}=g(\mathbf{z},\theta)}^{\top} \nabla_\theta g(\mathbf{z},\theta)$, which rearranges to the transpose form as stated (assuming column vectors and Jacobian notation).

In the special case from (b) and (c), $g(\mathbf{z},\mu) = \mu + \sigma\mathbf{z}$ and $p(\mathbf{z}) = \mathcal{N}(\mathbf{0},\mathbf{I})$ (or equivalently, $g(\mathbf{x},\mu) = \mu + \mathbf{x}$ and $p(\mathbf{x}) = p_{\mu=0}(\mathbf{x}) = \mathcal{N}(\mathbf{0},\sigma^2\mathbf{I})$).

---

5. Tensor Rematerialization

(a) Number of `fwd` Operations

- The network is divided into two segments:
  - **Segment 1:** Layers 1–5 (stored activations: input and layer 5)
  - **Segment 2:** Layers 6–10 (stored activation: layer 5)

For each layer i, we need the input to layer i (i.e., the output of layer $i-1$). If this activation is not stored, we recompute it by starting from the nearest stored activation and running forward through the necessary layers.

Segment 1 (Layers 1–5):

- Layer 1: input is stored → no recomputation
- Layer 2: recompute layer 1 → 1 `fwd`
- Layer 3: recompute layers 1–2 → 2 `fwd`
- Layer 4: recompute layers 1–3 → 3 `fwd`
- Layer 5: recompute layers 1–4 → 4 `fwd`

Total for Segment 1:

$$0 + 1 + 2 + 3 + 4 = 10 \text{ `fwd` operations}$$

Segment 2 (Layers 6–10):

- Layer 6: input is output of layer 5 (stored) → no recomputation
- Layer 7: recompute layer 6 → 1 `fwd`
- Layer 8: recompute layers 6–7 → 2 `fwd`
- Layer 9: recompute layers 6–8 → 3 `fwd`
- Layer 10: recompute layers 6–9 → 4 `fwd`

Total for Segment 2:

$$0 + 1 + 2 + 3 + 4 = 10 \text{ `fwd` operations}$$

Total `fwd` operations:

$$10 + 10 = 20$$

(b) Number of `loadmem` Operations

For each layer, we load the nearest stored activation:

- **Layers 1–5:** load the input (stored) → 5 `loadmem` operations
- **Layers 6–10:** load layer 5 (stored) → 5 `loadmem` operations

Total `loadmem` operations:

$$5 + 5 = 10$$

(c) Required Speed for `loaddisk`

Compare two methods:

1. **Tensor Rematerialization:**
   - 20 `fwd` operations × 20 ns = 400 ns
   - 10 `loadmem` operations × 10 ns = 100 ns
   - Total time: $400 + 100 = 500$ ns
2. **Disk Storage:**
   - Store all activations on disk
   - During backpropagation, load each activation from disk via `loaddisk`
   - Need to load outputs of layers 1–10 → 10 `loaddisk` operations
   - Let T be the time per `loaddisk` operation
   - Total time: $10T$

Set times equal:

$$10T = 500 \Rightarrow T = 50 \text{ ns}$$